WEB DEVELOPMENT · 3RD SEMESTER

Week 03 · Block 3c

Colour, Type & Spacing

<web-development> · Hochschule Ansbach
Matthias Kellnhofer

After this block, you can…

- Write colours four ways — **named, hex, rgb, hsl** — with transparency
- Choose the right **unit**, and say why `rem` for type and spacing
- Set up readable **typography**: a font stack, a type scale, line-height
- Apply **spacing** from a consistent scale
- Build a reusable colour & type **system** with CSS variables

You can select, size, and box any element.

Now make it look designed.

Colour, type, and spacing — applied *consistently* — are the difference between "works" and "intentional".

PART 1

Colour

Four ways to write the same colour

```
color: tomato;              /* 1. named — ~150 keywords   */
color: #ff6347;             /* 2. hex — #rrggbb           */
color: rgb(255 99 71);      /* 3. rgb — channels 0–255    */
color: hsl(9 100% 64%);     /* 4. hsl — hue, sat, light   */
```

- **hex** is the most common in the wild
- **hsl** is the friendliest to *adjust* — nudge lightness for hover states
- pick one style and stay consistent

Transparency

Add an **alpha** channel — `0` fully transparent, `1` fully opaque:

```
background: rgb(255 99 71 / 0.5);   /* 50% see-through */
background: hsl(9 100% 64% / 0.5);
```

This differs from the `opacity` property, which fades the **whole element** (text and all). An alpha channel fades only *that one colour*.

PART 2

Units

Absolute vs relative

- `px` — absolute; one fixed CSS pixel
- `%` — relative to the **parent**
- `em` — relative to the **element's** own font-size
- `rem` — relative to the **root** font-size *(the reliable one)*
- `vw` / `vh` — percent of the **viewport** width / height

Rule of thumb: **rem** for font-size and spacing, **px** for fine details like borders, **%** / **vw** for layout widths.

Why rem

```
:root { font-size: 16px; }   /* 1rem = 16px */

h1 { font-size: 2rem;   }    /* 32px */
p  { font-size: 1rem;   }    /* 16px */
```

- Everything is expressed **relative to one root size**
- Change the root → the whole page scales together
- It **respects the user's browser font-size** setting — an accessibility win `px` can't match

PART 3

Typography

Font families come in stacks

```
body {
  font-family: system-ui, -apple-system, "Segoe UI", sans-serif;
}
```

- The browser walks the list and uses the **first font available**
- Always end with a **generic family** (`sans-serif`, `serif`, `monospace`)
- This **system stack** is fast and native — no download

Want a specific typeface? **Web fonts** (Google Fonts, `@font-face`) let you ship one — at the cost of an extra download. A taste today; more when you need it.

A type scale, not random sizes

```
h1 { font-size: 2.5rem;  }
h2 { font-size: 1.75rem; }
p  { font-size: 1rem; line-height: 1.6; }
```

- Choose a **few** sizes that relate to each other — not `19px` here, `23px` there
- **`line-height` ~1.5–1.6** for body text → far easier to read
- Keep line length around **60–80 characters** (a `max-width` on your text)

PART 4

Spacing

Spacing is a design tool

From the box model (Block 3b): **padding** inside, **margin** outside. Used deliberately, they create rhythm.

```
.section { margin-bottom: 2rem;   }   /* gap between blocks */
.card    { padding: 1.5rem;       }   /* room inside a card */
```

- Pick spacing from a **consistent scale**: `0.5 · 1 · 1.5 · 2 · 3 rem`
- **Never** random values (`7px` here, `13px` there) — that's what reads as "unpolished"
- Consistent gaps between sections = visual rhythm

PART 5

A Consistent System

CSS variables tie it together

Define your palette, spacing, and type **once** in `:root`, then reuse them:

```css
:root {
  --color-text:  #14161b;
  --color-brand: #3a34d6;
  --space:       1rem;
}

body { color: var(--color-text); }
.btn { background: var(--color-brand); padding: var(--space); }
```

Why it matters

- **Consistency** — every element pulls from the same handful of values
- **Change once** — edit `--color-brand`, and the whole site updates
- **Readable** — `var(--space)` says *intent*; `16px` scattered everywhere doesn't
- Native CSS — **no build step**, works in every browser

> This is a **design system** in miniature. Real projects start exactly here: a small set of tokens, reused everywhere.

Recap

- **Colour**: named / hex / `rgb` / `hsl`, plus an **alpha** channel for transparency
- **Units**: `rem` for type & spacing (scales, accessible); `px` for details; `%` / `vw` for layout
- **Typography**: a font **stack**, a small **type scale**, generous **line-height**, sane line length
- **Spacing**: pick from a **consistent scale**; margins give rhythm
- **CSS variables** (`--x` / `var()`) keep the whole system consistent — define once, reuse

<EXERCISE> · 45 MIN

Your turn (At home) 👉

Give a plain page a **consistent colour & type system** — driven by CSS variables.

1. Define tokens in `:root` : a few **colours**, a **spacing** step, base **font**
2. Set a **type scale** in `rem` (`h1` , `h2` , `p`) with a readable `line-height`
3. Apply **spacing** from your scale — padding inside, margin between blocks
4. Reference everything via `var(--…)` — then change one token and watch it ripple

Starter: `exercises/week03/3c/` — open `README.md` . The page and an empty `:root` are ready for your system.